

Enums

Objectives

- What is an Enum?
- Simple Enums
- Adding properties and methods to Enums
- Switch / Case with Enums

The Need for Enums

- If you wanted variables to reference days of the week, how would you do it?
 - One approach is to use static final constants

```
public class WithoutEnums {  
    public static final int MONDAY = 1;  
    public static final int TUESDAY = 2;  
    public static final int WEDNESDAY = 3;  
    public static final int THURSDAY = 4;  
    public static final int FRIDAY = 5;  
    public static final int SATURDAY = 6;  
    public static final int SUNDAY = 7;  
}
```

Problem with Constants

- The problem with using public static final variables is that they are not type safe
 - The variable below called *today* can be set to any int value which makes no sense

```
int today = WithoutEnums.SUNDAY;  
// you could do this, but would be stupid  
today++;  
// what about this?  
today = -1500;
```

Using an Enum

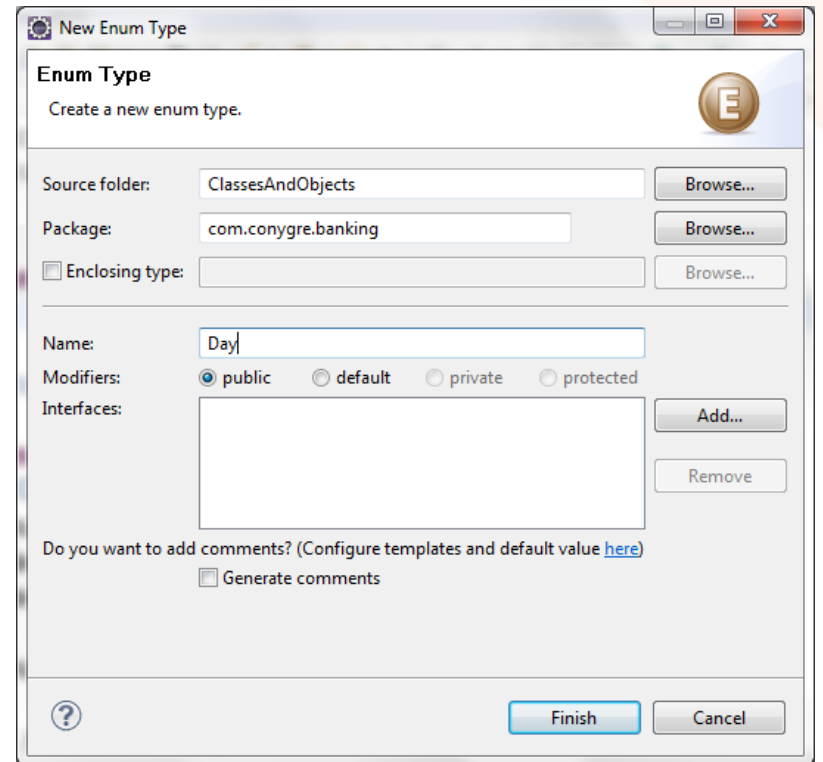
- An **Enum** can be created to represent the days of the week

```
enum Day {  
    MONDAY, TUESDAY, WEDNESDAY,  
    THURSDAY, FRIDAY, SATURDAY,  
    SUNDAY  
}
```

```
Day today = Day.SUNDAY;
```

Creating an Enum

- Enums can either be defined
 - In their own file like a Java class
 - Within a class like inner classes
- IDEs can be used to create Enums



An Enum with Properties

- It is also possible to define an Enum to have properties, constructors and methods
- Consider an Enum for a Month with
 - The name of the month
 - The number of days in the month
- The Enum will need a constructor for the number of days, and a method to return the number of days
- Enums have a values() method that returns all the Enum values

Enum Month Example

- Here is the Month Enum
- Note the constructor is private
 - Enums are not allowed public constructors
 - You cannot instantiate an Enum from outside

```
public enum Month {  
    // these are like constructor calls  
    January(31), February(28), March(31),  
    October(31), November(30),  
    December(31);  
  
    private int daysInMonth; // property  
  
    private Month ( int d ) { // constructor  
        daysInMonth = d;  
    }  
    public int numDays() { // method  
        return daysInMonth;  
    }  
}
```


Using the Month Enum

- Here is some code using the Month Enum

```
Month currentMonth = Month.February;  
System.out.println("This month has " + currentMonth.numDays());  
// use the values() method that all enums have  
for ( Month m : Month.values() ) {  
    System.out.println(m + ": " + m.numDays() );  
}
```

Switch / Case with Enum

- You may recall that the switch / case construct can be used with enums

```
switch (currentMonth) {  
    case December:  
        System.out.println("Christmas soon!");  
        break;  
    case January:  
        System.out.println("Sales begin this month");  
        break;  
    case February:  
        System.out.println("Romance in the air for valentines");  
        break;  
    default:  
        System.out.println("Not much going on this month");  
        break;  
}
```

Summary

- What is an Enum?
- Simple Enums
- Adding properties and methods to Enums
- Switch / Case with Enums